

Before you begin: DELETE THIS SECTION BEFORE SUBMISSION

- This template is for an original SoftwareX article. If you are submitting an update to a software article that has already been published, please use the software update template from the Guide for Authors.
- The format of a software article is very different to a traditional research article. To help you write yours, we have created this template. We will only consider software articles submitted using this template. Any submission that deviates widely will likely face desk rejection.
- It is mandatory to publicly share the code / software referred to in your software article. You'll find information on our software sharing criteria in the SoftwareX Guide for Authors.
- It's important to consult the Guide for Authors when preparing your manuscript; it highlights mandatory requirements and is packed with useful advice. Here are some key ones (please read):
 - We only accept GitHub repositories within Metadata Tables. This is required for forking to our SoftwareX repository if your paper is published. We are unable to accept other repositories.
 - GitHub repositories must have a well-documented README.md file, and a Licence.txt file. Your paper will be returned if these are missing.
 - Our word limit is 4,000 words. Please focus your paper on the software and place supporting information into your GitHub repository README.md file.
 - Limit figures to a maximum of six (6) and sparingly include any further display media.
 - We require the five (5) main templated sections outlined below. Your paper will be returned or rejected if these are missing.

Diff: data driven dependency injection framework for embedded systems

S. Niespodziany

*Institute of Computer Science, Warsaw University of Technology, plac Politechniki 1,
00-661 Warsaw, PL*

Abstract

Ca. 100 words diff = depondency injection framework First variant

Keywords: keyword 1, keyword 2, keyword 3

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0 - First variant
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/slalomir-niespodziany/diff
C3	Legal Code License	GPL-3.0
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C++14
C6	Compilation requirements, operating environments & dependencies	Platform-agnostic, requires C++ compiler (≥ 14)
C7	If available Link to developer documentation/manual	Documented example - https://github.com/slalomir-niespodziany/diff_broker
C8	Support email for questions	slalomir.niespodziany@pw.edu.pl

Table 1: Code metadata (mandatory)

1. Motivation and significance

Embedded systems are getting bigger and more complex as the technological advancements allow them to consist of more functionalities every day. Each system often comes in several variants, having customization options, subscription plans and update capabilities. Software is a vital part of each

embedded system and it grows accordingly to the system itself. While in the past it was sufficient to bootstrap and maintain a simple project for an embedded device, this is no longer the solution. It requires an up-front plan for configuration, enabling and disabling functionalities, adding new features not only at development stage, but also during the devices lifetime. The number of possible variants does not allow for cloning the project and deploying it with modifications each time. Such approach almost immediately breaks maintainability constraints. Another approach is to keep all the possible variants within a single project, but this fall short in the context of complexity, build-time, IP-management and CI/CD compatibility. The bigger the system, the more problematic customization of its software becomes.

Diff - *Dependency Injection Framework* - *First* variant - is a generic solution for embedded systems implemented using C++14 (and higher). It is a framework meant for embedded systems, intended for implementing modular applications with reusability and maintainability in mind. It is compatible with tools and methodologies widely used in embedded development. Its origin is nothing new. It was inspired by the mechanisms used in modern, scalable systems like the Spring framework. Its basic principle is to use a Json file controlling the topology and configuration of the resulting application. Such data-driven approach mimicks large-scale systems whose functionality is often defined via metadata files e.g. IaC, CI/CD. The highly utilized pattern of dependency injection allows framework engine to handle lifetime and dependencies of application building-blocks, called *components*. It allows for their loose coupling, which is critical maintainability and scalability.

1.1. Modular architecture

Diff imposes certain design decisions on the architecture of the implemented software. Their point is to make software modular. However, this is only a low level - physical - constraint, allowing full flexibility of the logical structure. The application is composed of components of two kinds. Firstly, *interface components* are lightweight and only define contracts. Secondly, more heavyweight, regular components, which provide implementations are simply called *components*. *Interface components* are provided in the form of abstract classes. They consist of API definitions, which dictate how *components* interact with each other. The word *component* operates at the logical level. Physically, components come in form of *modules*, which can be managed by external tools to support versioning and artifact management. In the context of C++ *modules* are simply header-only libraries (for *interface components*) or statically- or dynamically-linked libraries (for *components*). Figure 1 presents an example.

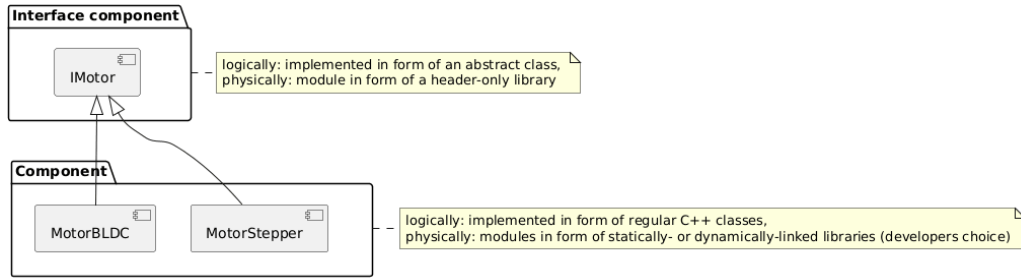


Figure 1: Exemplary set of components. *Interface component* IMotor provides an abstract class defining a contract. Its *module* comes in form of a lightweight header-only library. *Components* MotorBLDC and MotorStepper provide implementations for two types of motors, but also comply with the inherited interface to make them interchangeable. Their *modules* can be shipped in form of statically- or dynamically linked libraries.

Each component is wrapped in a single C++ class. For *interface components* this allows for deriving from them. For *components* this allows to create their instances. *Components* interact with each other by being injected through the *interface components* they implements. Such setup allows to create component instances according the application requirements defined by the Json file.

1.2. Loose coupling

While the architecture at the lowest level is imposed by the framework, the benefits it brings are as follows. While some of the listed arguments may seem very obvious in other software development areas, they are not necessarily industry-standards for embedded engineering.

1.2.1. Validation

Component tests are implemented and run in complete isolation from other components. Common unit test frameworks can be utilized for this purpose (e.g. GTest). Loose coupling allows for implementing most of the components using pure C++ - without any dependence on the platform or hardware. This opens the possibility to run and validate implementations during development - on host machines (e.g. x86 architecture) - and during deployment - within CI/CD pipelines, using containers.

1.2.2. Deployment

Each module is built at the time of its deployment, e.g. when it is submitted. The source code is immediately compiled and tested. Modules are built in form of libraries for all the required platforms up-front (e.g. x86, ARM).

From the same codebase, a building block becomes available for any environment which may be necessary. This can be automated at the CI/CD level. The modules are stored in artifact repository (e.g. Artifactory). This is useful for components intended for usage in multiple environments. A good example may be a communication protocol layer, utilized in both host and device environments, which differ significantly (different architecture, compiler, resources). Having in form of a library, which may be optimized for footprint on one architecture and for performance on the other allows for simply using an existing component in the following steps, rather than figuring how to rebuild (or worse - rewrite) an IP for another environment. This also reduces the build time of the resulting binary. As the modules are already compiled and available out-of-the-box. The only step left is to link them with the runnable, or even simpler - upload to the device, if the module comes as a shared library.

1.2.3. CI/CD compatibility

Because every *component* can be build independently from the others (apart from the lightweight *interface components*), any modern CI/CD pipeline can process it using standard tools. Toolchain for any target platform can be used within a dedicated container, used for the build process. Multiple containers provide can be used to build for multiple platforms. The process is easily parallelizable in cloud environment.

1.2.4. Versioning and multi-platform ability

The intrinsic compatibility with modern package management tools, such as Conan 2.0, allows for their usage for versioning and deploying modules for multiple platforms. Both *interface components* and regular *components* require versioning. While the first ones are deployed in form of header-only libraries, which makes them natively multi-pltform, the regular *components* need to be build for all the intended pltfoms separately. This can be easily automated with such tools and contenerization, where each container consists of a toolchain and configuration for a single platform and the source project is platform independent. Building the project within a selected container produces a binary for the corresponding platform.

1.2.5. Artifact management

Having multiple component versions for multiple platforms requires some organized means for storing the corresponding artifacts. One of the available tools is the Artifactory repository. It manages the created artifacts and makes them available for further usage. It is of crucial importance, that the selected package management tool shall easily integrate with it, to allows for

smooth dependency usage. For the Artifactory and Conan 2.0 this is true. It is particularly important that the artifact repository shall be a non-local resource. Firstly, for consistency - all the involved developers shall use the same repository. Secondly, and most importantly, the CI/CD pipeline shall perform component build and deployment. As this will most likely happen within some cloud environment, the artifact repo shall be accessible from there too.

1.2.6. Binary composition

The resulting binary shall be composed of multiple components. The build process must be able to pull and include the required ones. This is partially handled by the already mentioned tools. However, they only operate at the level of a module, while composing an executable for the given topology also requires the knowledge about the interactions between components - their dependencies, implemented interfaces configuration capabilities. This info is provided by the integrator of the runnable, but for two reasons an additional tool is required. Firstly, the Json file provided shall be validated as early as possible, or the binary may have problems at startup or may not build at all. Secondly, for the integrators convenience the list of available and compatible components and their versions shall be aggregated in one place. From there a convenient method to compose the desired topology shall also be provided. This is realized through a custom desktop application *diff-forge*.

1.3. Proposed tech-stack including diff

Diff framework alone would not be very usefull if not for the rest fo teh modern tech-stack. Its choice is a decision which for every project may be driven by differnet factors, thus resuting in different setup. A generic set of tools is proposed here as a starting point for deriving a project-specific toolchainn. However, the proposal is generic enough to allow for simply using it if there are no other constraints, limiting the use of the listed tools:

- diff - architecture and modularization,
- gcc/clang - build process,
- Conan - package management,
- Artifactory - artifact storage,
- Docker - contenerization,
- GitHub - source storage,

- GitHub Actions - CI/CD.

It is worth emphasizing, that nowadays many embedded projects are still executed as they were 20 years ago. This resembles using a single SVN repository, with separate branches for multiple software variants, built manually using an IDE in Windows-based developers PC. This is one end of the scale and there are obviously cases in the whole spectra - from the dysfunctional ones (as in this example) up to the well managed, adapting the newest trends and methodologies. The point is, however, that there are little industry standards, compared to e.g. webapp development, where typical problems are solved at higher levels, allowing the work to focus on application specific issues.

1.4. Physical and logical structure

Diff imposes physical structure - not for the application, but rather the environment, where it is developed and deployed. Nevertheless, this has some transitive impact on the internal applications architecture. From this perspective the app is composed of multiple separate components - which is in fact the main point. From there the app can be further architected into component groups. This may resemble layered architecture, hexagonal, or any of the designers choice. One of the possibilities, especially for embedded systems, where certain level of variability is required, is as follows - the design is split into three layers:

- applicaion
- system
- platform

Further sections explain this using the example of permanent storage requirement.

1.4.1. Application layer

This is a layer where the application logic resides. It is mainly driven by the business requirements, not necessarily depending on the used MCU/SoC, or the overall ssytem architecture. It may obviously assume certain restrictions e.g. on the underlying hardware, but does not define it specifically.

The app may require some way to store data permanently, it may define the size fo required sotrage, its non-functional parameters, like access time, durability, but does not define anything specific.

1.4.2. *System layer*

This layer fulfills the requirements imposed by the application layer, providing certain resources to it. The difference is however, that the top layer only specifies the requirement for certain use-cases and this layer specifies how they are provided. This is crucial for system flexibility.

This layer defines what is the target device providing the required functionality. It may be a flash-memory communicating over SPI bus if it fulfills the requirements falling down from the upper layer.

1.4.3. *Platform layer*

This specifies how the software integrates with the platform it is running within. This is crucial for software portability.

This layer provides the integration method of the driven device with the underlying platform. E.g. it provides the SPI driver to allow for communicating with the selected SPI-flash.

1.4.4. *Flexibility provided*

Now, if the memory chip unavailable (out-of-stock, or not produced anymore), the design may incorporate another similar chip. Then the only place where the software requires an update is the System layer - the module responsible for handling the specific chip can be replaced. Then if two software variants are deployed they only differ in that specific component.

If the replaced part is the platform the software runs on (e.g. the CPU/MCU), the only place where the design requires changes is the Platform layer - in this case particularly the SPI driver.

2. **Software description**

Diff is a header-only library imported into users project, in particular it is utilized by every *component*. It provides a set of template classes used for creating components, allowing them to be instantiated and managing the created topology. It allows to access the instantiated components from outside the created topology.

2.1. *Component implementation*

A *component* is implemented in form of a C++ class. To make the framework aware of it, it derives from a CRTP template `diff::ComponentT $\mathcal{J}$` provided by the diff library. A component may both depend and be a dependency for other components.

2.1.1. Consuming a dependency

To utilize functionalities of other components, a common contract - *interface component* - firstly needs to be defined in form of an abstract C++ class. It defines the required functionality, but does not specify the implementation. A *component* consumes a dependency through its constructor (Figure 2). It accepts a reference to an abstract interface class and uses its functionality utilizing polymorphism provided by C++.

2.1.2. Exposing a dependency

To expose its functionality, the component derives from *interface components* implemented as abstract classes. Usual inheritance, with no side-effects, is still available. To make the interface exposed, it shall be designated with *as <>* trait and declared within the *diff :: Component <>* template. This creates the inheritance relation and notifies the framework about the exposed interface. The following example depicts the possibilities:

```
class MyComponent :
    public diff::Component<MyComponent,
                        diff::as<ExposedInterface1>
                        diff::as<ExposedInterface2>>,
    public OtherClass {
public:
    MyComponent(ConsumedInterface & firstDependency) {
        ...
    }
};
```

Figure 2: Component visible for the framework uses a CRTP template class *Component <>*. Interfaces through which the component is exposed as a dependency shall be abstract classes and are declared within that template with *as <>* traits. Usual inheritance is also possible - the framework is not aware of it. In this example the component is exposed through *ExposedInterface1* and *ExposedInterface2* interfaces, while it simply inherits after the *OtherClass* - without any side-effects. TODO CONSTRUCTOR

An instance created by the framework always has a unique identifier of type `std::string` assigned by the user. When the component exposes interfaces they become available as dependencies for other components and are too identified by the same id. This is possible because an interface can only be inherited once by a C++ class. While this makes dependency identification simple, it has a serious disadvantage. A component may not expose multiple interfaces of the same type this way. For this reason *diff* provides a method for exposing multiple internal objects as dependencies. This is realized through a *side* member function of the following signature. It is intended to return references to the internal objects and id suffixes for each. The resulting id

for each such dependency is composed with the owning instance id and each suffix concatenated with an underscore character.

```
using namespace diff;

class MyComponent :
    public Component<MyComponent, ..., side<MyInterface>> {
public:
    ...

    void side(std::map<std::string, std::reference_wrapper<MyInterface>> &sideDependencies) {
        ...
    }
};
```

Figure 3: Component visible for the framework uses a CRTP template class *Component* <>. Interfaces through which the component is exposed as a dependency shall be abstract classes and are declared within that template with *as* <> traits. Usual inheritance is also possible - the framework is not aware of it. In this example the component is exposed through *MyFirstInterface* and *MySecondInterface* interfaces, while it simply inherits after the *OtherClass* - without any side-effects.

Because of the lack of reflection mechanism in C++14, the framework is notified about side-dependencies through an additional template argument using *side* <> trait.

2.1.3. Registering a component

2.2. Wrapping in a module

A *component* may provide implementations for arbitrary number of *interface components*. Thus it can be then injected as a dependency of one of the implemented interface types. It implements these interfaces either directly or exposes internal objects which implement them. The framework must obtain their references to be able to inject them as dependencies for other components. For this reason a *component* derives from a CRTP template class *Component_{ij}*, which informs the framework about the interfaces it implements directly and indirectly. The template also then defines a factory for the component. Traits *as_{ij}* and *side_{ij}* are used to designate interfaces implemented directly and indirectly. Indirectly implemented interfaces are referenced by the framework through a member function of the following signature:

```
void Component::side(std::map<std::string, std::reference_wrapper<Interface>> &
```

An exemplary component differs a bit, compared to a pure class implemented out of the context of diff. However the impact is not significant and can if required - a mock can be introduced to make the framework behavior a *no-op*.

```
class MyComponent : diff:Component<MyComponent, as<>>
```

Acknowledgements

Optionally thank people and institutes you need to acknowledge.

References

- [1] M. Fowler, “Inversion of Control Containers and the Dependency Injection Pattern,” martinowler.com, Jan. 2004. [Online]. Available: <https://martinfowler.com/articles/injection.html>
- [2] R. Johnson *et al.*, *Pro Spring*. Apress, 2005.
- [3] Microsoft, “Dependency injection in .NET,” Microsoft Learn Documentation, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection>
- [4] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison-Wesley, 2002.

Current executable software version